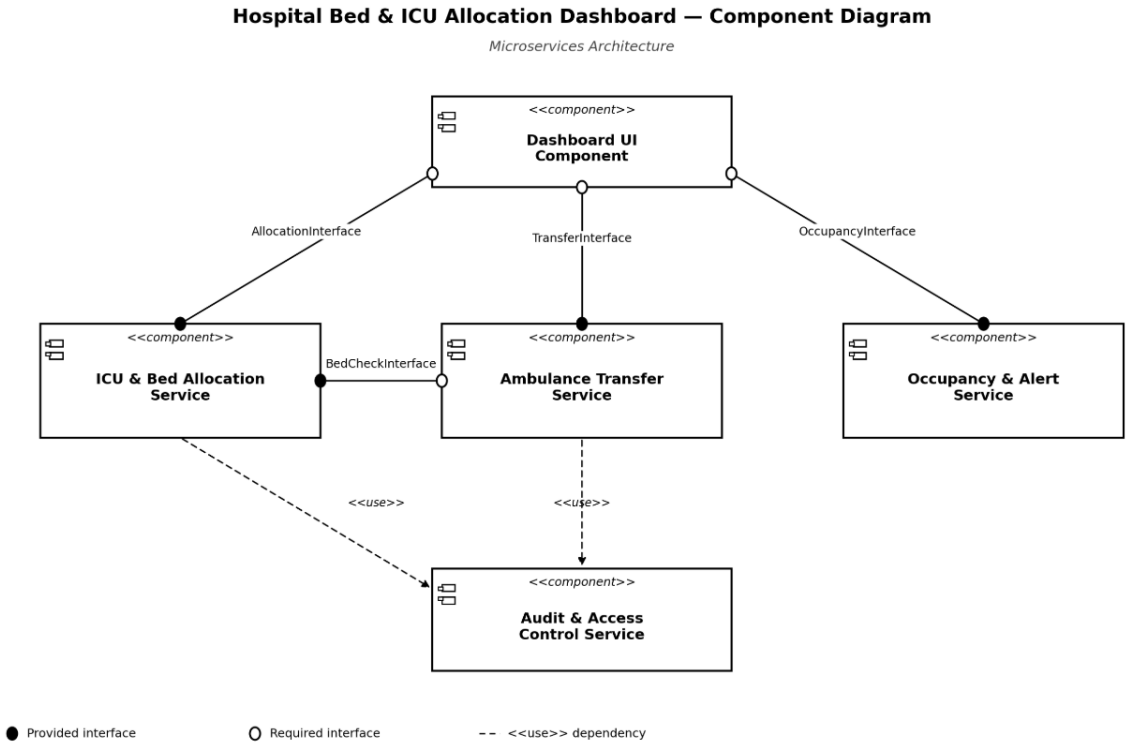
SE LAB 3

Name: Arnav Sharath Hanchanur

SRN: PES2UG24CS084

Section: B

Component Diagram:



Architecture Selection:

We chose Microservices Architecture for the Hospital Bed & ICU Allocation Dashboard System.

Reason 1: Independent Scaling Under Surge Load

During mass-casualty events or peak-admission periods, the triage severity scoring and ICU allocation logic (FR-001, FR-002) faces far heavier and more time-critical load than administrative features such as the occupancy reporting view (FR-005). Microservices let us scale the Triage & ICU Allocation Service on its own, without having to scale the entire application together the way a single-deployment layered system would require.

Reason 2: Fault Isolation for a Life-Critical System

The system serves actors with very different failure tolerances. An outage in the Ambulance Transfer Service or the Capacity Alert Service must never stop a Triage Nurse from updating bed status or stop the allocation engine from placing a critical patient (FR-002, FR-003). Because each capability is a separately deployable service behind the API Gateway, a fault in one service does not cascade into the others unlike a layered architecture, where a failure anywhere in the stack can bring down the whole request path.

Security Advantage

NFR-002 requires role-based access control and a full audit trail for every bed-status change, allocation, and transfer. Centralizing this in one dedicated Audit & Access Control Service, which every other service calls before a privileged action (confirming a transfer, overriding an allocation), means access rules and the audit log are enforced and recorded in exactly one place rather than re-implemented and potentially inconsistently inside each service. It also lets us patch or hardens that single service without redeploying the rest of the system.

Performance Benefit

NFR-001 requires bed-status changes to reach every connected dashboard client in under one second. Isolating this responsibility inside the API Gateway & WebSocket Broadcast Service lets that one service be optimized purely for low-latency fan-out (persistent WebSocket connections, in-memory pub/sub) without being slowed down by heavier operations happening elsewhere, such as the Bed Management Service's database writes or the Ambulance Transfer Service's calls to outside facilities.